

Instituto Politécnico Nacional

Escuela Superior de Cómputo

Web Client and Backend Development Framework.

Reporte de Integración BackEnd y FrontEnd con Spring Security y
Persistencia en Base de Datos

Profesor: M. en C. José Asunción Enríquez Zárate

Alumno: Luis David Martínez Martínez

ld.martinez.117@gmail.com

7CM3

22 de Junio, 2026

Contents

1	Introducción	1
2	Objetivo de la práctica	2
3	Conceptos	3
3.1	<i>Autenticación y Autorización</i>	3
3.2	<i>BCrypt y Hashing de Contraseñas</i>	3
3.3	<i>Políticas CORS (Cross-Origin Resource Sharing)</i>	3
4	Descripción del sistema	4
5	Modelo de base de datos	5
6	Desarrollo e Integración	6
6.1	<i>Configuración de Seguridad en Spring Boot</i>	6
6.2	<i>Carga de Detalles de Usuario desde la Base de Datos</i>	6
6.3	<i>Integración en la SPA (Front End)</i>	6
7	Resultados e Integración	7
8	Conclusión	8
9	Referencias Bibliográficas	9

List of Figures

1	Visualización del catálogo de habitaciones disponibles en la interfaz web.	7
2	Listado de reservaciones de un cliente autenticado, permitiendo modificar fechas o cancelar. . . .	7
3	Panel administrativo exclusivo para usuarios con ROLE_ADMIN.	7

List of Tables

1	Relaciones y tablas en base de datos MySQL	5
2	Criterios de Evaluación Práctica 3	8

1 Introducción

La integración de arquitecturas web modernas se basa habitualmente en el desacoplamiento de capas, donde el Back End opera como un proveedor de servicios RESTful autónomo y el Front End actúa como un cliente enriquecido (Single Page Application - SPA) que gestiona la experiencia del usuario y las vistas.

Para lograr que esta comunicación sea segura, el Back End debe contar con un framework de seguridad robusto como Spring Security, configurando filtros de autenticación y autorización capaces de validar el acceso según roles de usuario persistidos en base de datos. En esta práctica se documenta la finalización e integración del módulo de reservaciones del sistema hotelero, así como las estrategias aplicadas para establecer un canal seguro entre ambas capas.

2 Objetivo de la práctica

Completar el módulo de reservaciones y realizar la integración del Front End con los servicios expuestos por el Back End en Spring Boot, asegurando las peticiones mediante Spring Security con usuarios y roles cargados desde una base de datos relacional MySQL.

3 Conceptos

3.1 *Autenticación y Autorización*

La autenticación valida la identidad de un usuario (¿quién es?), típicamente cotejando credenciales cifradas en base de datos. La autorización determina los privilegios del usuario autenticado (¿qué recursos puede acceder?) comparando sus roles con las reglas de negocio establecidas.

3.2 *BCrypt y Hashing de Contraseñas*

BCrypt es una función de hashing de contraseñas adaptable y segura que incorpora un factor de costo y un valor salt aleatorio por cada password, impidiendo ataques de fuerza bruta o diccionario basados en tablas Rainbow.

3.3 *Políticas CORS (Cross-Origin Resource Sharing)*

Mecanismo de seguridad de los navegadores web que restringe a las páginas realizar peticiones HTTP hacia un dominio diferente al que originó la página original. El Back End debe estar configurado explícitamente para admitir cabeceras y métodos HTTP provenientes de los puertos del Front End.

4 Descripción del sistema

La aplicación web, denominada **ReservaEasy**, gestiona el flujo de reserva de habitaciones de hotel. Ofrece soporte para dos perfiles:

1. **Administradores (ROLE_ADMIN):** Crean, modifican y eliminan habitaciones, y gestionan las reservaciones globales de todo el hotel.
2. **Clientes (ROLE_USER):** Consultan cuartos disponibles, registran reservaciones a su nombre y cancelan o modifican sus reservaciones.

5 Modelo de base de datos

La base de datos relacional almacena la información organizada bajo las entidades descritas en la Tabla 1.

Tabla	Propósito	Atributos Principales
rooms	Catálogo de habitaciones	id, tipo, numero, precio
usuarios	Datos y acceso de usuarios	id, username, password, email
roles	Listado de roles autorizados	id, nombre (ROLE_USER, ROLE_ADMIN)
usuario_roles	Tabla asociativa N:M	usuario_id, rol_id
reservaciones	Registro de reservas	id, cuarto_id, usuario_id, estado

Table 1: Relaciones y tablas en base de datos MySQL

6 Desarrollo e Integración

6.1 Configuración de Seguridad en Spring Boot

En SecurityConfig.java se inyecta el codificador BCrypt y se restringen los métodos HTTP basados en los roles persistentes:

```
1  @Configuration
2  @EnableWebSecurity
3  public class SecurityConfig {
4      @Bean
5      public PasswordEncoder passwordEncoder() {
6          return new BCryptPasswordEncoder();
7      }
8
9      @Bean
10     public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
11         http.cors(cors -> cors.configurationSource(corsConfigurationSource()))
12             .csrf(csrf -> csrf.disable())
13             .authorizeHttpRequests(auth -> auth
14                 .requestMatchers("/api/v1/auth/**").permitAll()
15                 .requestMatchers(HttpMethod.GET, "/api/v1/cuartos/**").permitAll()
16                 .requestMatchers(HttpMethod.POST, "/api/v1/cuartos/**").hasRole("ADMIN")
17                 .requestMatchers(HttpMethod.GET, "/api/v1/reservaciones").hasRole("ADMIN")
18                 .anyRequest().authenticated()
19             )
20             .httpBasic(Customizer.withDefaults());
21         return http.build();
22     }
23 }
```

6.2 Carga de Detalles de Usuario desde la Base de Datos

Se implementó CustomUserDetailsService.java para leer credenciales de la base de datos:

```
1  @Service
2  @RequiredArgsConstructor
3  public class CustomUserDetailsService implements UserDetailsService {
4      private final UsuarioRepository usuarioRepository;
5
6      @Override
7      public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
8          Usuario usuario = usuarioRepository.findByUsername(username)
9              .orElseThrow(() -> new UsernameNotFoundException("Usuario no encontrado"));
10         return new User(
11             usuario.getUsername(),
12             usuario.getPassword(),
13             usuario.getRoles().stream()
14                 .map(r -> new SimpleGrantedAuthority(r.getNombre()))
15                 .collect(Collectors.toList())
16         );
17     }
18 }
```

6.3 Integración en la SPA (Front End)

En el archivo app.js del Front End se enlazan los formularios REST adjuntando las credenciales codificadas en Base64 en el encabezado de autorización HTTP:

```
1  const authHeader = 'Basic ' + btoa(username + ':' + password);
2  const rooms = await fetch('http://localhost:8090/api/v1/cuartos', {
3      headers: { 'Authorization': authHeader }
4  });
```

7 Resultados e Integración

La aplicación Front End cuenta con un diseño dinámico oscuro que reacciona según la sesión del usuario.

Figure 1: Visualización del catálogo de habitaciones disponibles en la interfaz web.

Al registrar una reserva, se ejecuta una petición POST al backend. El sistema verifica la disponibilidad del cuarto en las fechas indicadas, calcula el costo total, guarda el registro en la base de datos y envía automáticamente un correo de confirmación.

Figure 2: Listado de reservaciones de un cliente autenticado, permitiendo modificar fechas o cancelar.

Si el usuario inicia sesión con una cuenta que tiene el rol de administrador, la interfaz habilita la pestaña “Panel Admin”, permitiendo agregar habitaciones y ver las reservas de todos los clientes de la base de datos.

Figure 3: Panel administrativo exclusivo para usuarios con `ROLE_ADMIN`.

8 Conclusión

La integración de una SPA en HTML/CSS/JS con una API robusta en Spring Boot y Spring Security demuestra la viabilidad de arquitecturas desacopladas modernas. El uso de Spring Security con usuarios y roles persistidos permite aislar la lógica de permisos en el Back End, garantizando que el sistema sea seguro incluso ante posibles intentos de vulneración en las vistas del cliente web.

Criterio	Puntos	Realizado
Reservaciones (Back End)	30%	30%
Integración Front/Back	25%	25%
Spring Security	25%	25%
Persistencia DB	10%	10%
Reporte y Documentación	10%	10%
Total	100%	100%

Table 2: Criterios de Evaluación Práctica 3

Luis David Martínez Martínez

9 Referencias Bibliográficas

References

- [Spring Security, 2026] Spring Projects. *Spring Security Reference Architecture Guide*. Broadcom. Disponible en:
<https://docs.spring.io/spring-security/reference/>
- [CORS MDN, 2026] MDN Web Docs. *Cross-Origin Resource Sharing (CORS)*. Mozilla Network. Disponible en:
<https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>